QR Presence Check-in System: Development, Testing and Rollout Plan

Plan for a Telegram-based attendance system for a programming school with about 50 shared Linux PCs, generic wiped student profiles, Ansible-managed machines and about 50 students checking in at random times. Students confirm presence by scanning a QR code on the PC screen; staff track attended hours in an admin web UI.

1. Goals, constraints and non-goals

Goals

- Record, per student, when they arrived and when they left.
- Make it impractical to check in without being physically at a school PC (a photo of the QR sent to someone at home must not work).
- Require almost no effort from students after the first lesson.
- Give staff an admin UI that shows each student's attended hours per day, week and month, with corrections and exports.
- Be cheap to run and easy to roll out to all PCs with Ansible.

Fixed constraints

- PCs use generic student profiles that are fully wiped on restart, so a PC session proves only that someone is at that machine, not who.
- Telegram is already the only communication channel for all students, so the phone side is a Telegram bot: no app and no website for students.
- Accounts are local and managed with Ansible.
- An existing script, `/usr/local/bin/idle_logout`, logs students out after 30 minutes of inactivity (warning at 25, checks every 30 seconds).
- Phones use mobile data and cannot reach the school LAN.
- There is no proper Wi-Fi infrastructure, so nothing may depend on it.
- The study area is open 24/7 and nobody watches the system overnight. Staff check it at the start of each workday, so the system must restart itself and recover from failures without help.

Non-goals

- Defeating a determined attacker running a live relay of the screen. The system aims at deterrence plus detection, not perfection.
- Payroll-grade or exam-grade assurance.
- Tracking what students do on the PCs.

2. Architecture and the reasons behind it

2.1 Overview

There are five parts:

1. **PC agent** (the revised `idle_logout` script, deployed by Ansible). Runs inside each student session. Reports session start, heartbeats with idle time, and session end.
2. **QR display page** (`/qr`). Opened as the browser homepage on each PC. Shows a short-lived QR code that encodes a Telegram link carrying the token. Reachable only from school PC addresses.
3. **Telegram bot.** Receives the token when a student scans, and the enrollment code during first-time linking. Telegram attaches the sender's user ID to every message and students cannot forge it, so the bot is the phone-side identity.
4. **Check-in server.** Holds the database, validates scans, tracks PC sessions, closes visits and computes hours. Runs the bot and the admin UI. Needs outbound internet access only.
5. **Admin web UI.** For staff, on the school LAN, behind a login. Shows live attendance, hours per student and corrections.

2.2 Why this shape

Decision	Reason
The QR is shown on the PC screen, not on a wall or tablet	The token then exists only on a screen inside the school. A photo is stale within seconds, and an identity-free PC still proves location.

Decision	Reason
QR rotates every 30-60 s and is single-use	Shrinks the window in which a forwarded photo can work. Single use stops two people from sharing one scan. The window is a little wider than a plain web flow because Telegram adds a second or two of delivery time.
Student identity is the Telegram account	Telegram gives the bot a verified user ID with every message, and all students already use it. No cookies, no browser mismatch, no student-facing website, and a new phone costs nothing.
`/qr` and `/event` are LAN-only; the bot only makes outbound calls	The QR page is the location proof, so only school PCs may reach it. The bot polls Telegram, so no public endpoint, domain or certificate is needed.
Server tracks PC sessions as `(host, session_start)`	Gives a clean unit for check-out: when the session ends, all presence rows tied to it close. Works naturally with wiped profiles.
Reuse the idle logout script as the reporting agent	It already measures idle time and runs at every login, so there is one thing to deploy and no new daemon.
Server-side timeout as a fallback	Power-offs, crashes and network drops send no event. Without a timeout, visits would stay open forever.
Store raw events, compute visits and hours from them	Rules will change (grace periods, rounding, pair programming). Raw events let you recompute history, and every total in the admin UI can be traced back to events.
Record check-out at last activity, not at logout time	Otherwise every idle logout credits 30 phantom minutes.
Admin UI on the same server, corrections stored separately	One deployment to maintain. A manual correction is an audited entry (who, when, why) and never overwrites the raw record.

2.3 Where the server runs (a choice to make)

With Telegram as the phone channel, the server needs outbound internet only: the bot polls Telegram, and the PCs and staff browsers reach the server over the school LAN. No public address, domain or tunnel is required.

- **Option A: the admin PC.** No cost and the simplest network, since everything stays on the LAN. Weakness: if the PC is off or rebooted, check-ins stop, so it must stay on during opening hours and the database needs backups.
- **Option B: a small always-on machine** (mini PC or Raspberry Pi, roughly $50-150) on the wired network. Same design as A, but separate from daily admin work and easy to keep powered. Needs a fixed LAN address.
- **Option C: a small cloud server.** Survives school power or LAN problems, but then `/qr` and `/event` must be restricted to the school's public IP, which can change.

Recommendation: Option B, with Option A as the starting point for the pilot. A fixed LAN address and a nightly backup matter more than the hardware.

3. Core processes

3.1 Enrollment (first lesson, once per student)

1. Admin imports the student list and generates one single-use enrollment code per student, each as a personal link `https://t.me/<bot>?start=<code>`.
2. The student opens the link on their phone and presses Start.
3. The bot receives the code together with the student's Telegram user ID. The server validates the code (exists, unused, unexpired), marks it used and stores the link from that user ID to the student.
4. The bot replies "Linked as <name>". From now on that Telegram account is that student.

Why one-time codes and not asking students to type their name: otherwise anyone could claim someone else's identity. Send each link privately (a slip or a direct message), never in the group chat, because whoever opens a link first gets that identity.

3.2 Check-in (any time a student sits down)

1. Student logs into a PC. The agent sends `session_start` and begins heartbeats.

2. The homepage `/qr` identifies the PC by its IP. If that host has no live session, it shows "log in first". Otherwise it shows a rotating QR encoding `https://t.me/<bot>?start=<token>`, where the token is a short signed value (host, time window, nonce; at most 64 characters).
3. The student scans it with the phone camera, which opens Telegram on the bot with the token.
4. The bot receives the token and the sender's Telegram user ID. The server maps the ID to a student and verifies that the token is current, unused and belongs to a host with a live session.
5. It inserts a presence row `(student, host, session_start, confirmed_at)` and the bot replies "Checked in at PC-17".

3.3 Check-out (automatic by default)

- **Idle logout:** the script reports the event just before logging out. Check-out time is the receive time minus reported idle milliseconds.
- **Manual logout:** a `trap` on TERM/HUP reports the event (best effort).
- **Crash or power-off:** no heartbeat for about 3 minutes, so the server closes at the last heartbeat minus idle.
- **Optional:** a `/checkout` command (or button) in the bot chat for students who want an exact leave time.
- **Nightly cleanup:** the area is open 24/7, so there is no end of day. A job at a quiet hour (for example 04:00) closes any visit that is still open and flags it as auto-closed.
- **Midnight:** a visit that crosses 00:00 is split there for daily totals, so each day shows the time actually spent on that day.
- **Feedback:** the bot sends a short message such as "Checked out at 14:52, idle logout", so students see how their time was recorded.

All open presence rows tied to a session close when that session ends.

3.4 Administration

- Import or update the student list; generate and reissue enrollment codes.
- Unlink a Telegram account (lost account, wrong link) and issue a new code.
- Correct a visit and review the anomaly list (see 3.5 and section 5).

3.5 Admin UI: tracking student hours

Hours are computed from presence rows, never typed in, so every number traces back to a scan and a check-out. The admin UI is a server-rendered web page on the school LAN, behind a staff login.

What counts as an hour: time from the confirmed scan to the check-out time (last activity, not the logout moment). A visit that crosses midnight is split at 00:00 so daily totals are correct. A new scan on another PC closes the earlier visit, so overlapping visits for one student do not double-count. Rounding, minimum visit length and daily caps are policy choices (section 10.2).

Screen	What staff see and do
Today	Live list of PCs with a session, who checked in on each, and PCs with a session but no check-in
Hours	Students by day, week or month: total hours, number of visits, first arrival, last departure; filter by group; CSV export
Student detail	Every visit with start, end, PC and how it ended (idle logout, logout, timeout, auto-closed, manual); add, edit or delete a visit with a required reason
Anomalies	Odd scan latency, one student on several PCs, many students on one PC, auto-closed visits
Students	Import list, enrollment status, generate code, unlink Telegram account

Manual corrections are saved as separate entries with who, when and why. Original records stay intact and totals can always be recalculated.

4. Component design notes

4.1 Server

- One Go program built on the standard library (`net/http`), with SQLite through `modernc.org/sqlite` (pure Go, no C compiler needed). SQLite is enough for 50 users; move to PostgreSQL only if you outgrow it. Use WAL mode and a busy timeout, and keep write transactions short.

- The Telegram bot runs in the same process as a goroutine using long polling (for example `go-telegram/bot`; pin the version). Keep the bot token in a root-only file, never in the repository.
- Data model: `students`, `enroll_codes`, `telegram_links`, `sessions`, `presence`, `corrections`, plus an append-only `events` table for raw input.
- QR tokens: signed with HMAC (`crypto/hmac`, compared in constant time) over `host | time_window | nonce`, kept within Telegram's 64-character limit for start parameters. The server remembers used nonces for a few minutes to enforce single use.
- Internal endpoints authenticate with a shared host key and an IP allowlist.
- Time handling: always compute times from the server's own clock, never trust client timestamps.
- Admin UI: server-rendered pages behind a staff login with hashed passwords, reachable only from the LAN (details in 4.5).
- Deployment: cross-compile for the Pi (`GOOS=linux GOARCH=arm64`, or `arm` for a 32-bit system), copy one binary and run it under systemd. Keep the previous binary next to it, so rollback is a rename and a restart.

4.2 PC agent (revised `idle_logout`)

- Add `report session_start`, `report heartbeat <idle_ms>` every 1-2 cycles, `report idle_logout <idle_ms>` before logout, and a `trap` for manual logouts.
- Send a heartbeat even when the idle call returns nothing, so a flaky call is not read as the PC going offline.
- Use `flock` to prevent duplicate instances, and log via `logger` instead of `/tmp`.
- Replace the broad `pkill -KILL -u $USER` fallback with a session-scoped `loginctl terminate-session`.
- Deploy with an Ansible role: the script, the autostart entry, the host key (from Ansible Vault), the server URL and the browser homepage setting.

4.3 QR display page

- Plain HTML plus a small JS QR library, refreshing the code every 30-60 s and encoding the Telegram link. Add a visible countdown so students scan promptly.
- Run in the normal browser (homepage) or an always-on-top window. The student must be able to see it without hunting for it.

4.4 Telegram bot (phone side)

- `/start <code>` links a Telegram account to a student; `/start <token>` from a scan checks the student in.
- Clear replies: “Checked in at PC-17”, “Code expired, scan again”, “Log in on the PC first”, “Your account isn’t linked, ask staff”.
- `/checkout` and `/status` (current visit and today’s hours); an automatic message when a visit closes.
- One Telegram account per student and one student per account. The bot answers only in private chat and ignores groups.

4.5 Admin UI

- Server-rendered pages with `html/template` and htmx for small interactions, embedded in the binary (`embed`). No separate frontend build and no Node toolchain on the Pi.
- Staff accounts with bcrypt-hashed passwords (`golang.org/x/crypto/bcrypt`) and a session timeout. Teachers get a read-only role; admins can edit and correct.
- Hours are queried from `presence` plus `corrections`; the CSV export (`encoding/csv`) uses the same queries so the numbers always match the screen.
- Every correction and every unlink is written to an audit log.

5. Security model

Threat	Mitigation	Residual risk
Photo of the QR sent to someone remote	Short-lived, single-use token tied to a host	Relay within seconds is possible
Live relay (friend streams the screen)	Server logs scan latency and flags slow or odd scans	Cannot be fully stopped; acceptable for school use
Student scans for an absent friend (friend’s Telegram account open on the phone at that screen)	Bot replies with the PC name; account sharing is flagged by anomalies	Hard to detect automatically; use spot checks

Threat	Mitigation	Residual risk
Student links Telegram as someone else	One-time codes sent privately; one account per student	Wrong or shared link; unlink and reissue
Telegram account lent or logged in on another phone	Anomaly flags (one student on several PCs); admin can unlink	Same limit as lending a phone
Forged events from a PC	Host key plus IP allowlist; server accepts events only from known hosts	Key is readable under the shared profile, so “session alive” can be faked, but not identity
Bot token leak	Root-only file, rotate through BotFather	Brief impersonation until rotated
Admin UI misuse	Staff logins, LAN-only, roles, audit log of corrections	Trusted staff acting wrongly

Privacy: store only name, Telegram user ID, attendance times and staff corrections. State in writing what is collected and how long it is kept, limit admin UI access to the people who need it, and check local data-protection rules, especially if any students are minors.

6. Development plan

Rough effort is about 1.5-2.5 weeks to a pilot-ready build for one full-time developer using AI coding agents (section 6.1), or about 4-6 weeks part-time without them; both are estimates. Each phase ends with something demonstrable.

Phase 0: Decisions and setup (2-3 days). Settle the open choices in section 10, create the bot with BotFather, decide where the server runs, and set up a repository and a staging setup.

Phase 1: Server skeleton (about 1 week).

- Database schema and migrations.
- `/event` endpoint with host-key auth; session tracking and timeout job.
- Visit computation from raw events.
- Unit tests for the session and visit logic with scripted event sequences.

Phase 2: QR and Telegram check-in flow (about 1 week).

- `/qr` page with token generation and rotation.
- Bot handler for scan tokens: validation, single-use enforcement and presence insertion.
- Reply and error messages (expired token, not linked, no live session).

Phase 3: Enrollment and Telegram linking (3-5 days).

- Code generation from a student CSV and `/start <code>` handling in the bot.
- One account per student, unlink and relink, with an admin script for these until the UI exists.
- A short student instruction message to send in Telegram.

Phase 4: PC agent and Ansible role (about 1 week).

- Revised `idle_logout` with reporting and traps.
- Ansible role with vault-stored key, autostart entry and browser homepage.
- Test on one lab PC, then on a small group.

Phase 5: Admin UI, hours and anomaly detection (1-1.5 weeks).

- Staff login, Today and Hours screens, student detail with audited corrections, CSV export.
- Anomaly list: unusual scan latency, one student on many PCs, one PC with many students, auto-closed visits.

Phase 6: Hardening and documentation (3-5 days).

- Rate limiting for bot commands, log rotation, admin login hardening.
- Run the server and bot as a systemd service with `Restart=always`, enabled at boot, and set the machine's BIOS to power on after a power failure.
- Automatic nightly backup job and the nightly cleanup job at a quiet hour.
- A small alert script that sends an admin a Telegram message if no heartbeat arrives for 15 minutes.
- One-page student instructions and one-page staff runbook, including the morning check.

6.1 Using AI coding agents

Use agents for the pattern-heavy work and keep a human on security, the hours rules and real-device checks. Rollout time with real students does not shrink, so the saving is in the build, not the pilot.

Area	Use agents for	A human must own
Server and bot	Schema, migrations, endpoints, bot handlers, reply texts	Token signing, host-key check, Telegram link rules (read line by line)
Admin UI	Pages, filters, CSV export, login scaffolding	Roles, password handling, who can see and edit what
Hours logic	Implementation, once the tests exist	The rules (rounding, caps, overlaps) and the expected values in the tests
Tests	Unit, integration, replay, simulator and load scripts, edge cases	Deciding expected results; checking that tests assert real behavior
PC side	Draft `idle_logout` changes, Ansible role, systemd units	Running it on real lab PCs (GNOME, Wayland, wiped profiles)
Docs	Draft student instructions and the staff runbook	Following the steps on a real phone and PC to confirm they work

Working rules

- Commit this plan to the repository as the spec, plus a short project-instructions file with the stack (Go standard library, SQLite through a pure-Go driver, `html/template` pages) and rules such as server clock only and no new dependency without approval.
- Build one phase at a time. For hours and token logic, write or approve the tests first, then let the agent implement.
- Read every diff that touches tokens, links, session hooks or the admin login; small commits make this practical.
- Keep the stack plain so the generated code stays readable and fixable by staff later.
- Use fake data and a separate test bot while developing. Never paste the real bot token, host key or student data into a prompt.
- Pin module versions in `go.mod` and check the real documentation for the bot library, since agents can use outdated or invented calls.
- Ask a second agent session with fresh context to review the security-sensitive code against the threat table in section 5, then have the developer decide what to change.
- Record the assumptions the agent made in each pull request so the developer can confirm or reverse them.

Main risks of agent-written code

- Plausible but wrong hours logic that looks correct in a demo.
- Tests that only repeat the implementation, so they pass even when the rule is wrong.
- Outdated or invented library calls, extra dependencies and creeping complexity.
- Secrets or student data ending up in prompts or logs.

7. Testing plan

7.1 Automated

- **Unit tests, written first for hours and tokens:** table-driven `go test` cases for token signing and expiry, single-use enforcement, visit and hours computation (idle subtraction, timeouts, restarts, overlapping visits, corrections), Telegram link rules. Expected values come from a table of hand-computed sample sessions made by a human, not derived from the code.
- **Property and fuzz tests:** Go's built-in fuzzing for token parsing, and invariant checks for hours: recorded hours never exceed wall-clock time, visits for one student never overlap, closing a session twice changes nothing.
- **Integration tests:** replay of recorded event sequences against a test database, covering normal session, idle logout, power-off, double scan, two students on one PC, student switching PCs, restart mid-session.
- **Security tests:** expired token, token from another host, replayed token, forged host key, event from an unlisted IP, unlinked Telegram user, account unlinked by admin, enrollment code reuse, admin pages without login.
- **Static checks in CI:** `go vet`, `staticcheck`, `go test -race`, `govulncheck` for known-vulnerable dependencies and `gosec` for common security mistakes; the build fails on high-severity findings.
- **Checks on the tests themselves:** deliberately break a rule (remove the idle subtraction, accept an expired token) and confirm a test fails. Reject tests that only repeat the implementation's logic.
- **Independent review:** a fresh-context review of token, link and admin-login code against the threat table in section 5, followed by a human line-by-line read of those files.

7.2 Manual and field

- **Phone matrix:** test linking and scanning on at least two iPhones and three Android phones, including whether a scan opens the bot directly or shows a Start button, and a student with Telegram on several devices. This is where most real problems appear.
- **Network conditions:** phone on mobile data, on Wi-Fi without a route to the internet, with VPN or data saver on; Telegram slow or unreachable.
- **Failure drills:** unplug a PC, kill the network, stop the server mid-lesson, restart the bot. Confirm the system recovers and no visits stay open.
- **Recovery drills:** kill the server process and confirm systemd restarts it within seconds; power-cycle the machine and confirm the service and bot return without anyone logging in; confirm the alert script messages an admin.
- **Midnight and overnight:** run a visit across 00:00 and confirm it splits correctly in the Hours screen, run the nightly cleanup with open visits, and leave the system unattended for at least one night during the pilot, then compare the morning status with what actually happened.
- **Load test:** simulate 50 PCs sending heartbeats every 30-60 s plus a burst of 50 scans in a few minutes (start of lesson). SQLite should handle this easily; confirm it.
- **Clock test:** verify behavior when the server and a PC disagree on time (the server must rely only on its own clock).
- **Simulators:** agent-written scripts that fake 50 PCs and a burst of scans against a separate test bot are good for load and replay tests, but they only cover the server.
- **Human-only checks:** the phone matrix, the real lab PCs (GNOME idle detection, autostart, wiped profiles) and the student instructions followed step by step. Generated tests cannot replace these, so schedule them as their own day.

7.3 Acceptance criteria

- 95% or more of students can link their account in under two minutes without staff help.
- A scan completes in under five seconds.
- No visit stays open after the nightly cleanup job.
- A photo of a QR code scanned more than two minutes after it was shown is rejected 100% of the time.
- Recorded check-out is within one minute of the real last activity for idle logouts.
- Weekly hours in the admin UI match a manual count of a test week within five minutes per student.

- A visit that crosses midnight appears split across both days with correct totals.
- After a killed process or a power cycle, the server and bot answer scans again within two minutes without manual action.
- Every hours and token test traces to a hand-computed example, and deliberately breaking a rule makes at least one test fail.
- No high-severity findings from the static checks, and a human has read every file that handles tokens, links, session hooks and the admin login.

8. Implementation and rollout plan

1. **Staging on one PC (week 1 of rollout).** Staff only. Fix surprises in the agent and the Ansible role.
2. **Pilot group (week 2).** One class, 10-15 students, with the old attendance method running in parallel. Compare records and the admin UI's hours daily, and include at least one unattended night.
3. **Enrollment day.** Send each student their link privately in Telegram (or hand out slips) during a lesson, with staff walking the room. Watch the Students screen as links are used and keep a list of students who have not linked.
4. **Full rollout (week 3-4).** Deploy the Ansible role to all PCs, switch the browser homepage, give staff their admin UI accounts and a short walkthrough, and announce the new routine. Keep the old method for one more week as a safety net.
5. **Handover.** Staff runbook covers: the 30-second morning check and when to restart, reissue an enrollment code, unlink a Telegram account, correct a visit or add a missed one, read hours and the anomaly list, restart the server and the bot.

Communication to students: a short Telegram message (log in, open the browser, scan the code, wait for the bot's confirmation) and a clear statement of what is recorded and why, including that staff see attended hours.

9. Operations after launch

- **Auto-restart:** run the server and bot as a systemd service with `Restart=always`, enabled at boot, and set the machine to power on after a power failure. A crash or outage then recovers in seconds without anyone present.
- **Morning check (about 30 seconds):** at the start of each workday staff open the Today screen and look at the last heartbeat per PC, the last scan and the bot's last reply. A gap overnight means an outage; restart if needed and enter any missed visits as

corrections with a reason.

- **Optional alert:** a small script that messages an admin in Telegram if no heartbeat arrives for 15 minutes or the bot stops answering, so an overnight failure is not silent. It is about 20 lines and worth adding.
- **Backups:** nightly automatic copy of the database file to a separate location, plus a safe copy of the bot token and the staff account setup.
- **Maintenance:** keep the student list in one place (the same `students.yml` your Ansible uses where possible); archive leavers rather than deleting. Do updates and restarts in a fixed low-traffic hour and tell students that scans may fail briefly.
- **Review:** after one month, inspect the anomaly list and the failed-scan log to decide whether stricter measures are needed.

10. Risks, issues and choices to make

10.1 Issues likely to occur

Issue	Why it happens	Response
Scan opens Telegram but needs an extra Start tap	Telegram shows a Start button for a bot the student has not started yet	Test early; tell students to tap Start once; keep the QR visible long enough
Students forget to scan	Habit takes time	Prominent QR, staff reminders at the start of lessons, Today screen listing PCs with a session but no check-in
Telegram slow or unreachable	External dependency	Token window of 30-60 s; manual correction in the admin UI; old method as backup
Lost phone or new Telegram account	Normal	Admin unlinks and issues a new code
Student shares or reuses an enrollment link	Links sent carelessly	Send links privately, one-time codes, unlink and reissue

Issue	Why it happens	Response
Server or LAN down during opening hours	Single server	Let the agent queue events and retry; manual correction later; keep the old method as a backup
Server or bot down overnight, unnoticed	The area is open 24/7 and nobody is watching	systemd auto-restart, power-on-after-failure setting, optional alert script, morning check; missed visits entered as corrections
Visits that cross midnight	Totals counted on the start day would be wrong	Split visits at 00:00 when computing daily and weekly hours
Heartbeats lost during network blips	Wired network or server issue	Timeout of 3 minutes with last-heartbeat close prevents false early check-outs from short blips
Wiped profiles remove the agent	If the profile reset also resets autostart	Deploy the agent and autostart system-wide (`/etc/xdg/autostart`), not in the home directory
Student scans for an absent friend	System cannot see who is at the PC	Phone shows the PC name, periodic seat spot checks, anomaly flags

10.2 Choices to make

1. **Where the server runs:** admin PC, small always-on machine, or cloud (section 2.3). Recommended: a small always-on machine, starting on the admin PC for the pilot.
2. **One open presence per student, or allow overlap?** Recommended: one; a new scan closes the old one.
3. **Two students on one PC session:** allow (pair programming) with a flag, or reject. Recommended: allow and flag.
4. **Is the `/checkout` command included in version 1?** Recommended: yes, it is only a bot command; automatic check-out stays the default.
5. **How hours are counted:** scan to last activity, with or without rounding (for example to 5 minutes), a minimum visit length and a daily cap. Decide before building the Hours screen.

6. **Who may use the admin UI and make corrections:** recommended read-only for teachers, edit rights for a small set of admins.
7. **Data retention:** how long to keep attendance and Telegram link data (for example, one academic year). Decide with whoever owns data-protection obligations.
8. **What counts as "present":** the moment of scan, or scan plus a minimum time at the PC?
9. **Late scans:** a grace period or a "late" flag after lesson start? This is a policy choice that changes the reports.
10. **Fallback method:** the process when a student has no phone, a dead battery or no Telegram access (staff manual entry with a reason recorded in the admin UI).
11. **Reporting key location:** accept a key readable under the shared profile (simpler, limited damage), or use a root-owned reporting service (better, more work). Recommended: start simple and revisit if anomalies appear.
12. **Midnight handling:** split visits at 00:00 (recommended, gives correct daily totals) or count the whole visit on its start day.
13. **Overnight monitoring level:** manual morning check only (acceptable for version 1) or add the Telegram alert script (recommended, small effort).
14. **Maintenance window and nightly cleanup hour:** pick a quiet hour (for example 04:00) and announce it.

10.3 Things deliberately left out of version 1

- BLE beacons or NFC tags (consider only if relay cheating becomes a real problem).
- Seat assignment (not useful with sit-anywhere).
- A student-facing website or app (Telegram covers the phone side).
- Integration with the school's grading or LMS (add once the data has proven reliable).

11. Suggested next steps

1. Decide where the server runs and create the bot with BotFather, plus a separate test bot for development.
2. Collect the student list in a CSV and decide how hours are counted (section 10.2, items 5 and 8).
3. Set up the repository with this plan as the spec and a short project-instructions file for the coding agent (section 6.1), and write the hand-computed hours examples before any hours code.

4. Build Phase 1 and Phase 2 first; this gives a working end-to-end demo on one PC within about a week with agent help. The Hours screen follows in Phase 5.
5. Schedule the pilot class, the enrollment day and a day of human-only device testing early, since they depend on people rather than code.